

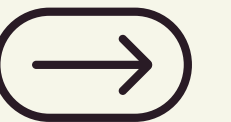


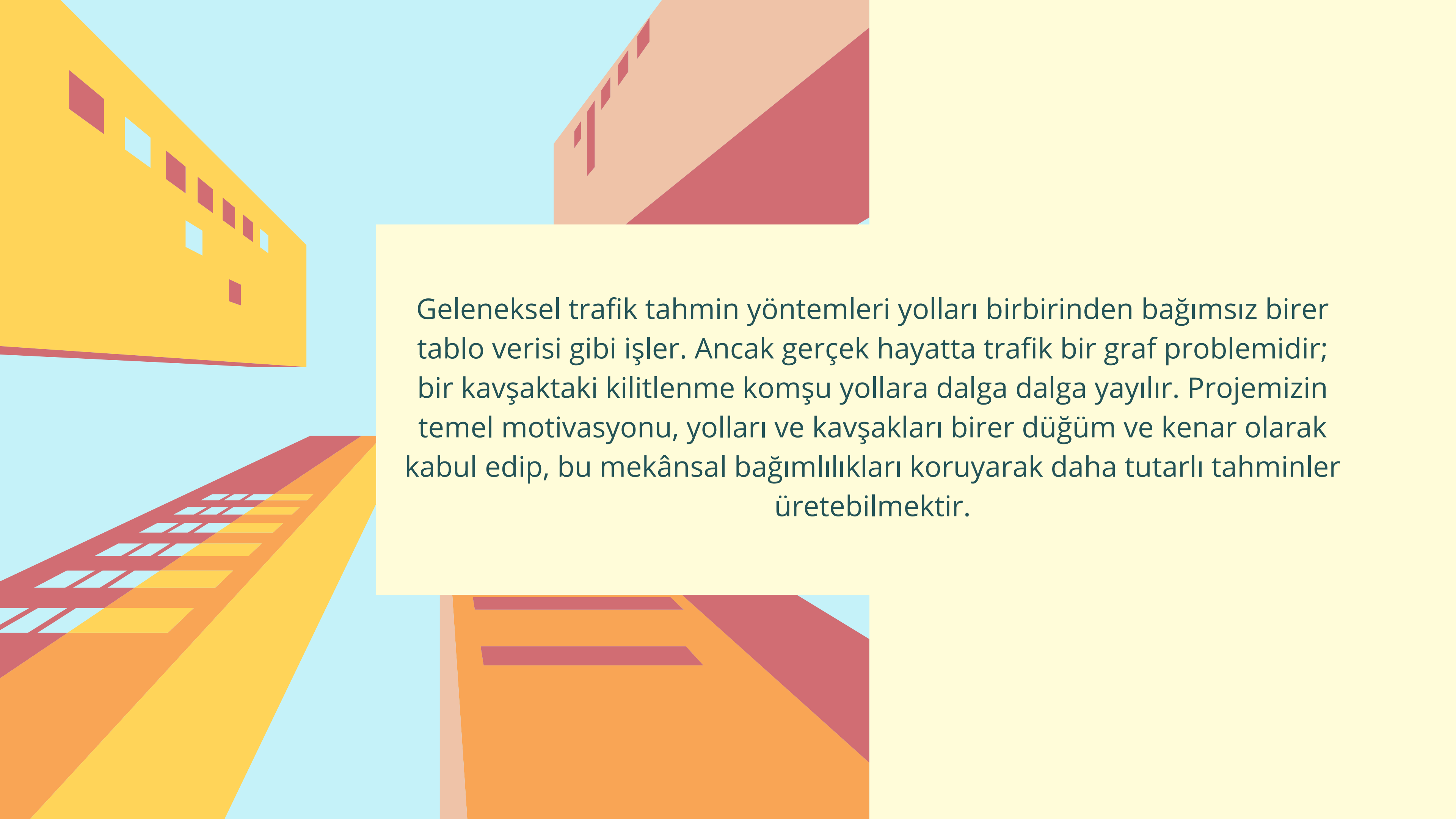
TRAFFIX: GNN-LSTM TABANLI İSTANBUL TRAFİK TAHMİN VE DARBOĞAZ ANALİZ SİSTEMİ

Proje ekibi: Onur Demir, Aymina Çakır, Volkan Çelebi, Egemen Bengi, Zeynep Yaren Çöçen

PROBLEM TANIMI VE MEVCUT SINIRLANDIRMALAR

- Geleneksel Yaklaşımlar: Trafik verilerinin bağımsız zaman serileri olarak ele alınması.
- Göz Ardı Edilen Faktör: Yol ağının fiziksel bağlantıları ve komşu segmentler arası mekânsal etkileşimler.
- Gerçek Dünya Dinamikleri: Bir noktada oluşan tıkanıklığın kavşaklar ve ana arterler aracılığıyla bölgesel darboğazlara dönüşmesi.
- Çözüm Yaklaşımı: Trafik probleminin doğal bir graf problemi olarak modellenmesi (Yollar=Kenarlar, Kavşaklar=Düğümmler).





Geleneksel trafik tahmin yöntemleri yolları birbirinden bağımsız birer tablo verisi gibi işler. Ancak gerçek hayatta trafik bir graf problemidir; bir kavşaktaki kilitlenme komşu yollara dalga dalga yayılır. Projemizin temel motivasyonu, yolları ve kavşakları birer düğüm ve kenar olarak kabul edip, bu mekânsal bağımlılıkları koruyarak daha tutarlı tahminler üretebilmektir.

PROJENİN AMACI VE ÖZGÜN YÖNÜ

- Projenin Kapsamı: İstanbul'un en yoğun ticaret ve ulaşım aksı olan Beşiktaş, Şişli ve Kağıthane hattı.
- Hedef Değişkenler: İleri zamanlı hız değişimi ve araç yoğunluğu tahmini.
- Özgün Yönü (Spatio-Temporal Modelleme):
- Mekânsal (Spatial): Graf Sinir Ağlarının (GNN) message passing (mesaj iletimi) yaklaşımıyla komşu düğümlerden bilgi toplaması.
- Zamansal (Temporal): LSTM katmanının geçmiş zaman pencerelerindeki trafik davranışlarını öğrenmesi.



Projemizin odak noktası İstanbul'un can damarı sayılan Beşiktaş, Şişli ve Kağıthane hattıdır. Çalışmamızın özgün tarafı, hibrit bir Spatio-Temporal mimari kullanmasıdır. Graf Sinir Ağları yolların fiziksel bağlantılarını öğrenirken, LSTM katmanı ise zaman serisi örüntülerini yakalamaktadır. Böylece hem mekânı hem zamanı aynı anda optimize edebiliyoruz.





VERİ SETİNİN HAZIRLANMASI VE COĞRAFİ FİLTRELEME

- BBox (Bounding Box) Filtrelemesi: Ham verinin genel coğrafi kırılması.
- İlçe Bazlı Sınır Poligonları (OSMnx): BBox'ın getireceği deniz aşırı (Üsküdar/Kadıköy) ve komşu mahalle veri şişmesini önlemek için cerrahi sınır çizimi.
- Map Matching (Harita Eşleme): GPS noktalarının gerçek yol çizgileriyle (LineString) tersine eşleştirilmesi.

VERİ KAYNAKLARI VE KAYNAK KISITLARI

- Trafik Verisi: Kaggle İstanbul Trafik Yoğunluğu Veri Seti (2020-2024).
- Yol Ağı Verisi: OpenStreetMap (OSM) sunucularından OSMnx kütüphanesi ile çekilen canlı harita verisi.
- Kaynak Kısıtı: Yaklaşık 7 GB boyutundaki ham verinin RAM'i kilitleme riski.
- Mühendislik Çözümü: Büyük CSV dosyalarının chunk-based (parça parça) okunması. Bölgesel koordinat filtrelemesi ve son 6 aylık kayıtlara odaklanarak veri hacminin optimize edilmesi.



- Sistem mimarisi 6 temel aşamadan oluşur: Veri Toplama, Ön İşleme, Graf Oluşturma, Model Eğitimi, Darboğaz Analizi, Kullanıcı Arayüzü.
- Boyut İndirgeme: Doğrudan model girdisi olmayan GEOHASH, MINIMUM_SPEED, MAXIMUM_SPEED gibi sütunların elenmesi.
- Harita Eşleştirme (Map Matching): GPS koordinatlarının KD-Tree algoritması kullanılarak en yakın yol segmentine ve graf düğümüne kilitlenmesi.

GENEL SİSTEM MİMARİSİ VE VERİ İŞLEME HATTI



GNN + LSTM MODELİ İÇİN PYTORCH VERİ SETİ HAZIRLAMA

Bu aşamada trafik verisi, modelin eğitimi için PyTorch tensörlerine dönüştürüldü.

Yapılanlar:

- Haritadaki kavşaklar ve yollar çıkarıldı.
- edge_index kavşakların hangi yollarla bağlı olduğunu gösterir.
- edge_weight ise bu yolların uzunluğunu/ağırlığını gösterir.
- Trafik verisi [Zaman, Düğüm, Özellik] formatına çevrildi.
- Sonuçta gnn_lstm_dataset.pt dosyası oluşturuldu.

Kod önce haritadan kavşakları ve yolları alıyor. Sonra her kavşağa bir GNN ID veriyor. Trafik CSV dosyasından saat, gün, hız ve araç sayısı bilgilerini okuyup büyük bir 3 boyutlu matris oluşturuyor.

X = [Zaman, Düğüm, Özellik] : Her zaman anında, her kavşak için bu 4 özellik tutuluyor.

Zaman → DATE_TIME sütunundan

Düğüm → OSMnx haritasındaki kavşaklardan

Özellik → HOUR, DAY_OF_WEEK, NORM_SPEED, NORM_VEHICLES

LSTM zaman değişimini, GNN ise kavşaklar arasındaki yol bağlantılarını öğrenebilecek hale geldi.



CİHAZ SEÇİMİ VE EARLY STOPPING

Bu bölümde modelin hangi cihazda çalışacağını otomatik seçen ve en iyi modeli kaydeden yardımcı yapılar eklendi.

get_device() fonksiyonu modelin çalışacağı donanımı seçer:

- NVIDIA GPU varsa → cuda
- Mac Apple Silicon varsa → mps
- Hiçbiri yoksa → cpu

Amaç:

Modeli mümkün olan en hızlı cihazda çalıştırmak.

Early Stopping yapısı

Bu sınıf eğitim sırasında validation loss değerini takip eder.

Validation loss, modelin doğrulama verisindeki hatasını gösterir.

Ne yapar?

- Validation loss düşerse modeli kaydeder.
- Loss değerinde iyileşme olmazsa sayaç artar.
- Eğer model belirli bir süre boyunca iyileşmezse early stopping devreye girerek eğitimi durdurur.
- En iyi modeli best_gnn_lstm_model.pt olarak saklar.

Lazy Sliding Window Dataset

Trafik verisi, modelin geçmişe bakarak tahmin yapabilmesi için zaman pencerelerine ayrıldı.

Bu modülde:

- Her örnekte geçmiş window_size kadar trafik verisi alınır.
- Hedef değer olarak pencerenin hemen sonrasındaki zaman adımı seçilir.
- Tüm veri baştan oluşturulmaz; DataLoader ihtiyaç oldukça örnek üretir.
- Böylece bellek kullanımı azalır ve eğitim daha verimli hale gelir.

Bu yapı sayesinde model, geçmiş trafik akışını kullanarak bir sonraki zaman adımındaki hız ve araç yoğunluğunu tahmin etmeyi öğrenir.



GNN-LSTM MODEL MİMARİSİ

Trafik tahmini için GNN ve LSTM yapıları birlikte kullanıldı.

GCNConv katmanı kavşaklar arasındaki yol ilişkilerini öğrenir. Yani bir kavşaktaki trafiğin komşu kavşaklardan nasıl etkilenebileceğini modellemeye çalışır.

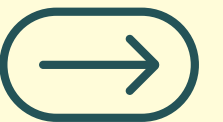
LSTM katmanı geçmiş zamanlardaki trafik değişimini öğrenir. Linear katman, GNN ve LSTM tarafından öğrenilen bilgileri kullanarak son tahmin değerlerini üretir. Bizim modelimizde bu çıktı bir sonraki zaman adımındaki hız ve araç yoğunluğudur.

Batch yapısı, modelin verileri tek tek değil gruplar halinde işlemesini sağlar. Böylece eğitim daha hızlı, daha kararlı ve bellek açısından daha verimli hale gelir.

Batch size = 16 ile model her adımda 16 tane trafik örneğini birlikte işler.

Modelin öğrenme mantığı

Yol ilişkileri + Zaman değişimi → Trafik tahmini



MODEL PERFORMANS METRİKLERİ VE BASELINE KARŞILAŞTIRMASI

Örneğin bir kavşakta son 3 saatteki hız değerleri :

08:00 → 42 km/s

09:00 → 38 km/s

10:00 → 35 km/s

Naive Baseline şöyle tahmin eder:

10:00 hız değeri = 35 km/s

11:00 tahmini = 35 km/s

GNN-LSTM :

11:00 tahmini = 32 km/s

Gerçek değer= 31 km/s

- Modelin hata oranını ölçmek için MAE metriği eklendi.
- Büyük tahmin hatalarını daha belirgin görmek için RMSE metriği eklendi.

Modeli basit bir yöntemle karşılaştırmak için Naive Baseline oluşturuldu.

Naive Baseline nedir?

Naive baseline, en basit tahmin yöntemidir.

Bu yöntemde model kullanılmaz.

Son zaman adımıdaki hız ve araç yoğunluğu, bir sonraki zaman adımı için aynen tahmin edilir.

Son bilinen trafik değeri = Bir sonraki tahmin

Neden baseline kullandık?

Modelimizin gerçekten işe yarayıp yaramadığını görmek için basit bir yöntemle karşılaştırma yaptık.

Eğer GNN-LSTM modeli baseline'dan daha düşük MAE ve RMSE verirse, model geçmiş veriden ve yol bağlantılarından anlamlı bir şey öğrenmiş demektir.

MAE ve RMSE hata metriğidir. Bu yüzden bu değerlerin küçük olması daha iyidir.



engine.py, GNN-LSTM modelinin eğitim ve test sürecini yöneten dosyadır. Bu dosyada iki temel fonksiyon bulunur:

1. Eğitim Fonksiyonu

`train_one_epoch()`

- Modeli eğitim moduna alır.
- Verileri batch batch işler.
- Tahmin üretir.
- Loss değerini hesaplar.
- Backpropagation ile hatayı geriye yayar.
- Optimizer ile model ağırlıklarını günceller.

2. Değerlendirme Fonksiyonu

`evaluate()`

- Modeli değerlendirme moduna alır.
- Validation ve test verisinde tahmin üretir.
- Model ağırlıklarını güncellemez.
- Loss, MAE ve RMSE değerlerini hesaplar.

Engine dosyası eğitim aşamasında model ağırlıkları güncellenirken, değerlendirme aşamasında model sadece tahmin yapar ve performans metrikleri hesaplanır.

EĞİTİM VE DEĞERLENDİRME MOTORU

train.py, hazırlanan trafik veri setini yükleyip modeli eğiten ana pipeline'dır.

Veri, zaman sırası bozulmadan kronolojik olarak %80 Eğitim, %10 Validation,%10 Test olarak ayrılmıştır.

Model her epoch sonunda validation verisiyle kontrol edilir ve en iyi sonucu veren model kaydedilir.

Son aşamada bu model test verisi üzerinde performans metrikleri ile birlikte değerlendirilir ve naive baseline ile karşılaştırılır.

GNN-LSTM EĞİTİM PIPELINE'I



İlk eğitimde GNN-LSTM modeli test verisi üzerinde tahmin üretti ancak naive baseline daha iyi sonuç verdi.

Model sonucu:

Test MAE: 0.001264

Test RMSE: 0.010882

Test R²: 0.656332

Naive baseline sonucu:

Baseline MAE: 0.000212

Baseline RMSE: 0.003862

Baseline R²: 0.956719

Yorum:

Model düşük hata üretse de baseline daha başarılı çıktı. Bunun nedeni, trafik verisinde ardışık zaman adımlarının birbirine çok benzemesidir. Yani “son bilinen değeri aynen tahmin etme” yöntemi kısa vadede çok güçlü çalıştı.

İLK EĞİTİM SONUCU: BASELINE PROBLEMİ



2.EĞİTİM İÇİN YAPILAN İYİLEŞTİRMELER

- **Early Stopping Delta Güncellendi**
- Küçük validation iyileşmelerini yakalamak için delta değeri 0.0001'den 1e-6'ya düşürüldü.
- **Hiperparametreler Düzenlendi**
- Model daha dengeli öğrensin diye gnn_hidden, lstm_hidden ve learning rate değerleri küçültüldü.
- **Edge Weight Normalize Edildi**
- Yol uzunlukları 0–1 aralığına çekildi; kısa yolların etkisi artırıldı, uzun yolların etkisi azaltıldı.
- **Prediction Horizon Eklendi**
- Modelin sadece bir sonraki zamanı değil, daha ileri trafik durumunu tahmin etmesi sağlandı.

İyileştirme Sonucu

Metrik	GNN-LSTM Modeli	Naive Baseline
MAE	0.000478	0.000546
RMSE	0.004465	0.008330
R ²	0.942164	0.798748

Yapılan iyileştirmelerden sonra GNN-LSTM modeli tüm metriklerde naive baseline'ı geçti. Özellikle RMSE değerinin düşmesi, modelin büyük tahmin hatalarını baseline'a göre daha başarılı şekilde azalttığını gösterdi.



3.EĞİTİM İÇİN YAPILAN İYİLEŞTİRMELER

- **Model kapasitesi değerlendirildi:**
 - Daha büyük GNN ve LSTM katmanları denenerek modelin daha fazla örüntü öğrenebilmesi hedeflendi.
- **Dropout yapısı eklendi:**
 - Modelin eğitim verisini ezberlemesini azaltmak ve daha genel öğrenmesini sağlamak amaçlandı.
- **Learning rate scheduler önerildi:**
 - Eğitim kaybı belirli bir noktada sabitlenirse öğrenme hızının otomatik azaltılması hedeflendi.
- **Veri zaman filtresi uygulandı:**
 - Trafik verisi yalnızca 2024 Temmuz sonrası kayıtlarla sınırlandırıldı.
- **Saatlik gruplama yapıldı:**
 - Aynı saat ve aynı kavşağa ait kayıtların ortalaması alınarak daha düzenli veri oluşturuldu.
- **Veri boyutu azaltıldı:**
 - Dataset yaklaşık 6 GB'dan 630 MB'a düşürüldü. Model daha güncel, sade ve daha az gürültülü veri üzerinde eğitildi.

Model sonucu:

Test MAE: 0.000983

Test RMSE: 0.008642

Test R²: 0.764457

Baseline sonucu:

Baseline MAE: 0.000567

Baseline RMSE: 0.008454

Baseline R²: 0.774597

Bu denemede model trafik değişimini öğrenebildi ve baseline'a oldukça yaklaştı. Train, validation ve test loss değerlerinin birbirine yakın olması overfitting olmadığını gösterdi. Ancak naive baseline az farkla daha iyi sonuç verdi. Bu da saatlik trafik verisinde ardışık zaman adımlarının hâlâ birbirine çok benzediğini ve son gözlemin güçlü bir tahmin referansı olduğunu gösterdi.



4.EĞİTİM İÇİN YAPILAN İYİLEŞTİRMELER

Yapılan ayarlar:

1. **Horizon:** 6 → 12
2. **Dropout:** 0.2 → 0.1
3. **Epoch:** 50

Model Sonucu

Metrik	GNN-LSTM Modeli	Naive Baseline
MAE	0.001059	0.000654
RMSE	0.007319	0.009356
R ²	0.830535	0.723053

Bu denemede model, RMSE ve R² değerlerinde baseline'dan daha başarılı oldu. MAE değerinde baseline daha iyi görünse de, modelin RMSE değerini düşürmesi büyük tahmin hatalarını daha iyi azalttığını gösterdi. R² değerinin 0.830535 olması da modelin trafik değişimini baseline'a göre daha iyi açıkladığını gösterir.



5.EĞİTİM İÇİN YAPILAN İYİLEŞTİRMELER

- Sadece gerçek veri bulunan hücrelerde hata hesaplandı.
- Eksik/boş hücrelerin model değerlendirmesini etkilemesi azaltıldı.
- Model ve baseline daha adil şekilde karşılaştırıldı.

Sonuçlar

Metrik	GNN-LSTM	Baseline
MAE	0.037133	0.058991
RMSE	0.052050	0.085991
R ²	0.764158	0.356306

Bu denemede GNN-LSTM modeli tüm metriklerde baseline'dan daha iyi sonuç verdi. Bu yüzden en güvenilir sonuç olarak maskeli değerlendirme tercih edildi.Masked loss sayesinde model yalnızca gerçek trafik verisi bulunan hücreler üzerinden değerlendirildi. Sonuçta GNN-LSTM modeli MAE, RMSE ve R² metriklerinin tamamında naive baseline'ı geçti.

GENEL KARŞILAŞTIRMA TABLOSU

Deneme	Temel Değişiklik	Model MAE	Model RMSE	Model R ²	Baseline MAE	Baseline RMSE	Baseline R ²	Sonuç
İlk eğitim	R ² + baseline eklendi	0.001264	0.010882	0.656332	0.000212	0.003862	0.956719	Baseline daha iyi
İyileştirilmiş model	Horizon + hiperparametre	0.000478	0.004465	0.942164	0.000546	0.008330	0.798748	Model daha iyi
Saatlik veri	W12 / H6	0.000983	0.008642	0.764457	0.000567	0.008454	0.774597	Baseline az farkla iyi
H12 / 50 epoch	Dropout + uzun eğitim	0.001059	0.007319	0.830535	0.000654	0.009356	0.723053	Model RMSE/R ² 'de iyi
Masked loss	Gerçek hücrelerde ölçüm	0.037133	0.052050	0.764158	0.058991	0.085991	0.356306	En güvenilir sonuç

VERİ KAYMASI VE YANLIŞ HARİTA GÖSTERİMİ

İLK MODEL TAHMİN ÇIKTISI



KAYDIRILMAMIŞ VERİ VE DOĞRU HARİTA



İLK MODELİN SAHİP OLDUĞU SORUNLAR:

- BELİRLENEN İLERİYE DÖNÜK ZAMAN BOYUNCA TAHMİN YAPMAMASI
- VERİSİ OLMAYAN YOLLARI ORTALAMA HIZLARA KAYDIRMASI
- BİRBİRİNE YAKIN HIZLARA SAHİP YOLLARA AYNI DEĞERLERİ ATAMASI

ÇÖZÜM?

- Normalizasyon sınırını değiştirmek
- LayerNorm ve Sigmoid Kapısı kullanmak
- Modelin, yolların kendi değerlerine daha çok güvenmesini sağlamak
- Sadece verisi olan sokakları haritada göstermek

MODELİ DÜZELTMEK VE SAHTE VERİ

Modelimiz tahmin yaparken üç farklı zaman penceresini besin olarak alır.

- Kısa Dönem: Son 3 saatteki anlık değişiklikler (örn: yeni olmuş bir kaza).
- Günlük Döngü: Dün tam olarak aynı saatteki akış. Yani bir gün önceki trafik yoğunluğunu dikkate alır.
- Haftalık Döngü: Geçen hafta aynı gündeki trafik yoğunluğunu dikkate alır.

Eğer hepsi işaretlenirse tüm seçenekleri karıştırır !

ÇOKLU AKIŞ
(MULTI-STREAM)

KONTROL PANELİ

1- Tahmin Ufku: Kullanıcı, 1 ila 6 saat arası gelecekteki durumu görebilir.

2- Isı Haritası: Kullanıcı, modelin tahmin ettiği değerleri harita üzerinden dinamik renkler yardımı ile görebilir.

3- Darboğazlar: Sadece tıkanmış yollar, çok kalabalık caddeler gibi bilgileri kullanıcıya gösterir.

Hız (km/s)	Harita Rengi	Anlamı
< 5.0 veya Nan	Silik Gri	Veri eksikliği veya yol kapalı
< 25.0	Koyu Kırmızı	Ciddi yoğunluk / Dur-kalk trafik
25.0 - 40.0	Turuncu	Yavaş akış
40.0 - 55.0	Sarı	Akıcı yoğunluk
> 55.0	Koyu Yeşil	Trafik tamamen açık

Kontrol Paneli

Tahmin Ufku (Saat)

1

Zamansal Bağlam Kolları

Yapay zekanın hangi pencereleri hesaba katacağını seçin:

- ☒ Kısa Dönem (Son 3 Saat şoku)
- ☒ Günlük Döngü (Dün Aynı Saat)
- ☒ Haftalık Döngü (Geçen Hafta)

Harita Modu

- ☒ Isı Haritası (Genel Trafik)
- ☐ Sadece Darboğazlar (Uyarı)

İstanbul Tıkanıklık Yayılımı (+1 Saat)



Gelişmiş Şehir Analitiği

İzlemek İsteddiğiniz Güzergahı Seçin:

Büyükdere Caddesi

1 Saat Sonra Büyükdere Caddesi Durumu

46.5 km/s

↑ Orta Seviye Akış



BİLGİ (Mavi): Büyükdere Caddesi üzerinde akış devam ediyor ancak yer yer hafif yavaşlamalar görülebilir. (Tahmini Hız: 46.5 km/s)